

Alex briefing — LG three-step pipeline (ASSIGN → SCORE → SELECT)

Purpose: Walk-through sheet for HAND17 / PD17 — formulas, notation, and one-line meanings.
Audience: Charles + Alex Gates
Last synced: 2026-08-13
Binding: `../.. / BINDING_Selection_is_its_own_step.md` — environment ≠ advancement; score ≠ select.
Alex-facing name: LG (Levine–Gates sim). Code paths still say `grandchild_*`.

Carry these three sentences

- 1. **ASSIGN** builds rosters (who sits with whom). **p** lives here only.
 - 2. **SCORE** ranks players: $S_i = \hat{A}_i - \lambda L_C$. **λ** lives here only.
 - 3. **SELECT** picks winners from that rank — **v1 = top K** (same winner rule whether $\lambda = 0$ or $\lambda > 0$).
- The Hero plot is an **outcome** (Layer A). It is **not** the scoring equation.

Pipeline at a glance

Player pool ($\hat{A}_i$) → **ASSIGN** (LG, ρ) → **Rosters** (μ_j) → **SCORE** ($S_i = \hat{A}_i - \lambda L_C$) → **SELECT** (top K) → Bin vs Hero

Step	Question	Main knobs	Output
ASSIGN	Who lands on which team?	p , roster caps	<code>pool_id</code> , team centroids $\hat{T}_j \equiv \mu_j$
SCORE	How do we rank?	λ , θ , γ	Selection score S_i (code column: <code>selection_weight</code>)
SELECT	Who wins?	K (slots)	<code>Y_selected</code> ∈ {0,1}

Step 1 — ASSIGN (LG model + how we build sim data)

What LG does (one sentence)

Players arrive one at a time (random order). Each player **draws a team** with probability proportional to **open seats × homophily toward that team’s current centroid**. After seating, the team centroid updates from **actual members only**.

Assignment weight (unnormalized)

For player i considering team j with $R_j > 0$ open seats:

$$\tilde{w}_{ij} = R_j e^{(-\rho|\hat{A}_i - \mu_j|)}$$

Seat probabilities over open teams: $P(j \mid i) \propto \tilde{w}_{ij}$ (normalize to sum to 1).

$$P(j \mid i) = \frac{\tilde{w}_{ij}}{\sum_{j'} \tilde{w}_{ij'}}$$

Symbol	Say it	Meaning
$\hat{A}_i$	“A-hat sub i”	Ability of player i . In PD17 LG runs: empirical PPM z within season from the locked hero panel (not a fresh random draw).
μ_j	“mu sub j”	Running team centroid — mean ability of players already on team j . Starts at league mean $\mu_0 = \hat{A}$.
R_j	“R sub j”	Remaining stub capacity on team j (open seats). Full teams get weight 0.
ρ	“rho”	Homophily knob (ASSIGN only). Higher $\rho \rightarrow$ sit near similar μ_j . $\rho = 0 \rightarrow$ seat choice $\approx$ random among open teams.
$\exp(-\rho \hat{A}_i - \mu_j)$	—	Soft match : small ability gap $\rightarrow$ high weight; large gap $\rightarrow$ low weight. Absolute value (L1), not squared.

First player on an empty team: $\mu_j \leftarrow \hat{A}_i$ (the league mean μ_0 is **not** treated as a pseudo-player).

After each seat: if team j had n members before player i arrived,

$$\mu_j \leftarrow \frac{n \mu_j + \hat{A}_i}{n + 1}.$$

Final roster mean (diagnostic label): $\hat{T}_j = \mu_j$ after ASSIGN completes.

Code: `sports/541_grandchild_homophily_assign.py` → `grandchild_assign`.

How we create LG sim data (PD17 apples-to-apples)

For each season $t = 2011, \dots, 2021$:

- 1. **Load abilities** — same filtered player-season panel as the empirical Hero (PPM z within season, min-minutes filter).
- 2. **Load roster caps** — exact **NCAA roster-size multiset** for that season: one capacity per real `(team_id, season)`, summing to N_t players.
No synthetic fixed $C = 15$ unless we explicitly run that variant.
- 3. **One LG realization** — run `grandchild_assign` with chosen $\mathbf{p}$ and those caps.
- 4. **Stack** player-seasons across seasons for distribution / SELECT readouts.

What is held from NCAA: ability draws $\hat{A}_i$, total N , per-team stub sizes, empirical $\mathbf{K}$ for SELECT (draft count).

What is synthetic: team labels `pool_id` (who sits with whom).

Code: `load_empirical_roster_caps_season`, `run_one_realization` in `541_grandchild_homophily_assign.py`; panel scripts under `sports/scripts/grandchild_*`.

ASSIGN diagnostics (not SCORE / not SELECT)

Measured **after** seating on a fixed partition:

Stat	Formula (plain)	Use
D	Mean $(\hat{A}_i - \mu_{g(i)})^2$ over players	Within-team spread
global_wss	$\sum_i (\hat{A}_i - \mu_{g(i)})^2$	Same gaps, raw sum
H_sort	1 – within-team SS/total SS	How much team labels explain ability spread

$\mathbf{p}$ is the generative knob; **H_sort** is the realized readout. Do not say “we set H_sort = 0.15.” Say “at $\mathbf{p} = 0.5$, realized H_sort $\approx$... on this run.”
H_sort is the fraction of league ability variance explained by team labels after assign (1 minus within-team SS over total SS).

Step 2 — SCORE

Alex score (ranking only)

$$S_i = \hat{A}_i - \lambda L_C$$

Symbol	Role
S_i	Selection score — ranks players for SELECT. Higher → better rank.
$\hat{A}_i$	Talent (same units as empirical PPM z in PD17).
λ	Congestion weight in the score (Alex λ). $\lambda = 0 \Rightarrow$ talent-only ranking.
L_C	Team congestion — defined below. Same value for every player on team j .

Binding: λ enters **SCORE**, not ASSIGN. Pass A / λ -sweep holds ASSIGN and SELECT fixed, toggles λ .

Prose vs code: say **selection score** S_i in the paper and with Alex. The repo column is `selection_weight` (legacy name from older stochastic winner rules that **draw** winners $\propto S_i$); function `selection_weights()` builds the same vector.

Code mapping: $\lambda \rightarrow$ `loo_gap_weight`; $S_i \rightarrow$ `selection_weight`; algebraically $S_i = \hat{A}_i - w \cdot L$ with $w \equiv \lambda$ when $L = L_C$.

Soft peer viability (building block)

For any player k with ability $\hat{A}_k$:

$$v_k \equiv \sigma(\gamma(\hat{A}_k - \theta)) = \frac{1}{1 + \exp(-\gamma(\hat{A}_k - \theta))}$$

Symbol	Meaning
$\sigma(\cdot)$	Logistic — soft “is this peer viable?”
θ	Viability cutline (sigmoid center). Not “median of selected.”
γ	Sharpness. Large $\gamma \approx$ hard step at θ ; small $\gamma \approx$ smooth ramp.

PD17 LG default in comparability scripts: $\gamma = 0.5$ (swept separately in characterization work). Teaching preset 539 often uses $\gamma = 10$.

Team congestion L_C (PD16 / Alex lock)

Congestion is a **property of the team**, not leave-one-out for scoring:

For player i on team $j = g(i)$, with roster $\{k : g(k) = j\}$:

$$L_C(i) = L_{C,j} = \frac{1}{|j|} \sum_{k \in j} v_k = \frac{1}{|j|} \sum_{k \in j} \sigma(\gamma(\hat{A}_k - \theta))$$

Include self in the team mean. Every teammate on team j gets the **same** $L_{C,j}$.

Interpretation: Average “viable-peer pressure” on the roster — how many strong peers crowd the team, measured softly via σ .

Contrast (visualization only): LOO congestion excludes player i from the mean — used for Hero **x-axis** (`poolq_loo`), not for PD16 team score. On slides: team L_C for SCORE; LOO pool quality for binning vs Hero.

Code: `add_team_pool_columns` → column `pool_c_smooth_team`; SCORE mode `crowding_smooth_team`.

Notation cheat sheet (hats, i's, j's)

Notation	Object	Layer
$\hat{A}_i$	Ability of player i	Input / measured talent
$g(i)$	Team index player i sits on after ASSIGN ($\in \{1, \dots, J\}$; code: <code>pool_id</code>)	After ASSIGN
$\mu_{g(i)}$	Centroid of player i's team — same as μ_j when $j = g(i)$; equals $\hat{T}_j$	After ASSIGN (diagnostic)
j	Generic team label (use when the formula is “for team j ”)	After ASSIGN
$\mu_j, \hat{T}_j$	Final roster mean on team j	After ASSIGN (diagnostic)
v_k	Soft viability of player k	SCORE ingredient
$L_{C,j}$	Team congestion on team j	SCORE ingredient
S_i	Selection score for player i (code: <code>selection_weight</code>)	SCORE output
L_Q / <code>poolq_loo</code>	LOO mean teammate $\hat{A}$	Hero bin axis (not in S_i for team- L_C runs)

How we set θ from K and N

One idea: the viability cutline θ is the ability level where the **top fraction** K/N of the pool begins — same selectivity as draft slots. Let $F_t(\cdot)$ be the **CDF** of within-season ability $\hat{A}$ in season t (empirical PPM z on the locked panel). Then:

$$\theta_t = F_t^{-1}\left(1 - \frac{K_t}{N_t}\right)$$

Same thing in words: **θ is the inverse CDF at probability $1 - K/N$.**

Code: `np.quantile(ability, 1 - K/N)`.

Step	What we use
K	Draft count that season (empirical <code>Y_draft</code> sum)
N	Player-seasons in the filtered panel
K/N	Selection rate (MBB $\approx 1\%$)
θ	Ability cutline in $\sigma(\gamma(\hat{A} - \theta))$

Equivalent readout: about K/N of players have $\hat{A}_i > \theta$ (top tail of the pool).

MBB example: $K/N \approx 0.01 \Rightarrow \theta \approx$ **99th percentile** of within-season PPM z — “viable peer” $\approx$ someone in the draft-competitive tail.

Per season: K and N (hence θ) can change year to year; we do not fix one global θ unless we say so.

Unit matching (when A is z-scored)

If $\hat{A}$ is z-scored and $L_C \in [0, 1]$, code may multiply L_C by a scale $\approx p90(\hat{A}) - p10(\hat{A})$ so $\lambda \cdot L_C$ can move ranks. Empirical PPM z panel: check `l_term_scale` / `CROWDING_L_Z_SCALE` in config. On **[0,1]** synthetic draws, no scale needed.

Step 3 — SELECT

Winner rule (v1)

Deterministic top K: within each season, sort all players by S_i (descending); set `Y_selected = 1` for the K highest scores.

$$Y_i = \mathbf{1}\{S_i \text{ is among the top } K \text{ scores in the season}\}$$

Symbol	Meaning
K	Slots (draft picks). PD17: empirical draft count per season , not a fixed gallery default.
K/N	Selectivity — system feature (MBB $\approx$ 1%; characterization gallery often 10%).
<code>Y_selected</code>	1 = selected / drafted in the sim; 0 otherwise.

Code: `choose_selected(..., winner_selection="C")` in `tier1_pool_assignment.py`.

Future (not v1): stochastic draft from scores — change **SELECT only**, leave S_i fixed.

What we plot after SELECT

- Bin players by **LOO pool quality** (`poolq_loo`) — same axis family as the empirical Hero.
- y-axis**: fraction with `Y_selected = 1` in each bin.
- Compare NCAA (real rosters, real draft) vs LG (sim rosters, top-K on sim scores).

λ -sweep finding (Aug 2026): inverted-U on LOO emerges when λ in **SCORE** is large enough (roughly 1.25–3+ in current LG caps runs), not from ρ or input caps alone.

Quick “do not merge” table

Often confused	Keep separate
ρ vs λ	ρ = ASSIGN homophily; λ = SCORE congestion weight
Score vs select	S_i selection score ranks; top-K picks
<code>selection_weight</code> vs S_i	Same object — use “selection score” in prose; <code>selection_weight</code> is the CSV column name
Team L_C vs LOO L_C	Team = SCORE (PD16); LOO = Hero axis / older Pass A arm
$\hat{T}_j$ vs T_{j^*}	LG centroid μ_j is endogenous ; legacy soft-assign used exogenous targets T_{j^*}
Hero curve vs S_i	Hero = empirical outcome; S_i = generative ranking rule
H_sort vs ρ	H_sort = realized sorting readout; ρ = ASSIGN knob

Suggested walk order (15–20 min)

- Three steps** — one slide; score $\neq$ select.
- ASSIGN** — weight formula + “empirical $\hat{A}$, synthetic seating, empirical caps.”
- Team L_C** — σ , θ , ν ; **one number per team**; write the team mean formula.
- θ from K/N** — quantile sentence + MBB $\approx$ 99th percentile.
- SCORE** — $S_i = \hat{A}_i - \lambda L_C$; λ sweep punchline if time.
- SELECT** — top K = empirical drafts; show overlay vs Hero.

Code & doc pointers

Topic	Location
LG ASSIGN engine	<code>sports/541_grandchild_homophily_assign.py</code>
SCORE / SELECT engine	<code>sports/tier1_pool_assignment.py</code> (§5 team L_C , §6 SCORE, §7 SELECT)
Defaults / knobs	<code>sports/tier1_sim_config.py</code>
Empirical vs LG L_C	<code>sports/scripts/grandchild_empirical_lc_compare.py</code>
λ sweep on SELECT	<code>sports/scripts/grandchild_lambda_select_sweep.py</code>
H_sort / D memo	<code>grandchild_assign/GRANDCHILD_D_and_H_sort_interpretation.md</code>
Three-layer story	<code>../02_Three_Kinds_of_Model.md</code>

Charles runs PDF from this file when ready: `./scripts/convert_single_md_to_pdf.sh 3-Master_Plan/re_entry/HEROs_and_PASSes/Alex_LG_three_step_briefing.md`